

AI Lecture Generation System

Complete Architecture & How It Works

*A simple guide to understand every part of the system —
from PDF upload to finished lecture video*



What Is This System?

This system takes any educational content — a PDF textbook, a PowerPoint slideshow, or a recorded lecture video — and **automatically converts it into a professional animated lecture video**, just like what you see on Coursera, Udemy, or Khan Academy.

What Goes IN	The Magic	What Comes OUT
• PDF textbooks • PPT slides • Lecture videos	14 intelligent processing steps powered by AI	• Animated video • AI narration • Auto subtitles

Complete Pipeline — 14 Steps at a Glance

1 Input Validation Checks file for errors	2 Extraction Layer Reads text from files	3 Content Cleaner Fixes junk text/OCR	4 Claude Planner AI builds lecture plan	5 Pydantic Validation Checks AI JSON output	6 TTS Narration Converts text to voice	7 Timeline Master Syncs audio + visuals
8 Redis Queue Manages job order	9 Scene Workers Splits work to machines	10 Manim + Remotion Creates animations	11 FFmpeg Composer Merges all into video	12 Validation Agent QA checks final video	13 R2 Storage Saves video to cloud	14 CDN Delivery Streams to students

Step-by-Step: Input to AI Planning

STEP 1

Input Validation Agent

Before doing anything, the system checks your uploaded file. Is the PDF readable? Is the video in a supported format? Are the slides corrupted? If something is broken, it stops here — saving expensive GPU time later.

Why it matters: Without this check, bad files would crash the entire pipeline after hours of work.

Tools used: pdftinfo, ffprobe, Pillow/OpenCV, Python validators

STEP 2

Extraction Layer

This step reads the actual content from your file. For digital PDFs it uses PyMuPDF (fast and accurate). For scanned/image PDFs it uses PaddleOCR to read text from images. For uploaded videos it uses OpenAI Whisper to transcribe spoken words into text.

Why it matters: We need the raw content before AI can do anything. Garbage in = garbage out.

Tools used: PyMuPDF (digital PDF), PaddleOCR (scanned PDF), Whisper (video transcription)

STEP 3

Content Cleaning Agent

OCR isn't perfect. It might output 'L4ngGr@ph' instead of 'LangGraph'. This agent fixes broken symbols, removes repeated junk text, fixes spacing, and repairs encoding issues so Claude receives clean, readable content.

Why it matters: Claude is smart but still produces better output when given clean text.

Tools used: Regex, NLP cleanup libraries, language correction

STEP 4

Claude Scene Planner (The AI Brain)

This is the most important step. Claude Sonnet 4 reads all the cleaned content and creates a complete lecture blueprint — deciding what narration to say in each scene, what animation to show, how long each scene lasts, where to add examples and pauses. It outputs structured JSON that all downstream steps rely on.

Why it matters: This is what makes the system intelligent. Without Claude, there is no lecture structure.

Tools used: Claude Sonnet 4 (claude-sonnet-4-20250514) via Anthropic API

STEP 5

Pydantic Validation Layer

Claude's JSON output is validated before anything renders. If Claude writes 'duration: ten seconds' instead of 'duration: 12', Pydantic catches it immediately and rejects the bad output — before the GPU even starts.

Why it matters: This single step eliminates ~90% of render crashes. It is cheap insurance against expensive failures.

Tools used: Python Pydantic library

Step-by-Step: Voice, Sync & Rendering

STEP 6

TTS Narration Generator

The narration script from Claude is converted into a real human-sounding voice. ElevenLabs is the top choice — it produces realistic voices with natural emotion and proper pauses. For a cheaper MVP, Google TTS with SSML (Speech Synthesis Markup Language) tags can control pauses and emphasis to sound less robotic.

Why it matters: Text-to-speech quality directly determines how professional the lecture feels to students.

Tools used: ElevenLabs (production), Google TTS + SSML (MVP/budget option)

STEP 7

Timeline Master Agent (The Movie Director)

This agent does NOT generate audio or animations. Its only job is synchronization. It measures the actual audio duration (e.g. 12.43 seconds), then tells the animation system exactly how long each visual should play. If audio is longer than the animation, it stretches transitions or adds pauses automatically.

Why it matters: Without sync, audio and visuals fall out of alignment — making the video feel broken.

Tools used: Custom timing logic, audio duration measurement tools

STEP 8

Redis Queue (Job Manager)

When multiple users upload files at the same time, Redis manages the queue. Think of it like a ticket system — each job gets a number, and workers process them in order. Redis also handles retries if a job fails, tracks progress, and reports job status. Used by OpenAI, Uber, Netflix, and Discord.

Why it matters: Without a queue, 5 simultaneous uploads would crash each other. Redis keeps everything orderly.

Tools used: Redis, Celery or RQ (Python worker frameworks)

STEP 9

Scene Workers (Parallel Processors)

Workers are like factory operators — they take jobs from Redis and run them on separate machines in parallel. A 15-scene lecture can be split: Worker 1 renders scenes 1-5, Worker 2 renders scenes 6-10, Worker 3 renders scenes 11-15 — all at the same time. This turns 4-8 hours of sequential rendering into 20-60 minutes.

Why it matters: Parallel workers are the key to making the system fast enough to be practical.

Tools used: Celery/RQ workers, distributed compute

STEP 10 — Animation Rendering: Manim + Remotion

Manim	Remotion
Math · graphs · code · AI flow diagrams Physics simulations · timelines Style: 3Blue1Brown educational videos	React-based web animations Modern UI/UX visuals · infographics Interactive-style motion graphics
<i>Best for: technical/science content</i>	<i>Best for: modern visual storytelling</i>

Both tools are used together. Manim handles technical animations (math, graphs, code flows). Remotion handles modern web-style motion graphics. GPU rendering on Modal (serverless, auto-scaling) runs the actual render jobs.

Important: Raw AI-generated Manim/Remotion code often fails. The system uses **templates** — Claude fills in variables like `render_graph(topic='LangGraph')` instead of writing full animation code. This makes rendering much more stable.

Step-by-Step: Assembly, QA & Delivery

STEP 11 FFmpeg Composer (Video Assembly)

FFmpeg is the industry-standard video tool used literally everywhere (YouTube, VLC, WhatsApp). It takes all rendered scene clips, the voice audio, subtitle files, and transition effects and merges them into one final MP4 video.

Why it matters: This is the assembly line — all the individual parts become one complete product.

Tools used: FFmpeg (open source, industry standard)

STEP 12 Validation Agent (Quality Check)

Before storing the video, an automated QA agent checks: Is audio in sync? Are there any dropped frames? Do subtitles appear at the right time? Are any scenes missing? Are there black frames indicating a render failure?

Why it matters: Without final QA, a broken video could be delivered to students.

Tools used: FFprobe, custom sync-checking scripts

STEP 13 Cloudflare R2 Storage

All generated files (scene clips, audio, subtitles, final video) are stored in Cloudflare R2 — an object storage system like Amazon S3, but with zero egress fees. This means you don't pay extra every time a student watches a video.

Why it matters: R2 saves significant cost compared to AWS S3, especially as the platform scales.

Tools used: Cloudflare R2 (S3-compatible object storage)

STEP 14 CDN Delivery (Global Streaming)

The final video is delivered to students anywhere in the world through a CDN (Content Delivery Network). CDN servers are located globally, so a student in India gets the video from a nearby server — fast load, smooth playback.

Why it matters: Without CDN, students far from the server experience slow, buffering videos.

Tools used: Cloudflare CDN (built-in with R2)

Complete Technology Stack

Category	Tool / Service	Why This Choice
Backend API	FastAPI (Python)	Fast, modern, async-ready
Frontend	Next.js (React)	Best for web apps
PDF Extraction	PyMuPDF	Speed + accuracy + formatting
OCR (scanned)	PaddleOCR	Better than Tesseract for tables/handwriting
Video → Text	OpenAI Whisper	95-98% accuracy, industry standard
AI Brain	Claude Sonnet 4	Best long-context + structured output
Output Validation	Pydantic	Prevents GPU crashes from bad JSON
Voice (Pro)	ElevenLabs	Most human-sounding TTS available
Voice (Budget)	Google TTS + SSML	Cheap, acceptable quality
Job Queue	Redis	Ultra-fast, used by Netflix/Uber

Workers	Celery / RQ	Python worker frameworks
Animation 1	Manim	Math, graphs, code, technical visuals
Animation 2	Remotion	React-based modern motion graphics
Video Merge	FFmpeg	Industry standard, free, powerful
GPU Rendering	Modal (serverless)	Auto-scale, pay-per-render, startup-friendly
Storage	Cloudflare R2	Zero egress fees vs AWS S3
CDN	Cloudflare	Global, fast, built into R2

Expected Performance & Known Risks

Performance Benchmarks

Metric	Expected Result	Notes
Digital PDF text extraction	95–99% accuracy	PyMuPDF on clean PDFs
Scanned/OCR document reading	85–92% accuracy	PaddleOCR with post-cleaning
TTS voice realism (ElevenLabs)	9.5 / 10	Near-human quality
Audio-visual sync drift	< 200 ms	Timeline Master keeps tight sync
20-minute lecture render time	20–60 minutes	With parallel workers + GPU
Render stability (templates)	Very High	Templates eliminate most failures

Biggest Risks & How We Solve Them

■ Manim render crashes	Use template system — Claude fills variables, not raw code. Pydantic validates before GPU starts.
■ Robotic-sounding voice	Use ElevenLabs instead of basic TTS. SSML markup for natural pauses with Google TTS.
■ Audio-visual sync drift	Timeline Master measures real audio length, then adjusts animation timing to match.
■ Huge GPU costs	Use distributed workers + smart caching. Modal serverless = pay only for actual render time.
■ OCR text garbage	PaddleOCR + Content Cleaning Agent fixes symbols, spacing, and encoding issues.
■ Long render times	Parallel scene workers split 15 scenes across 3 workers simultaneously.

In Summary

This system is a complete, production-grade AI pipeline. A teacher uploads a PDF. The system validates it, extracts content, cleans it, hands it to Claude which plans the entire lecture, validates the plan, generates voice, syncs everything, queues render jobs, renders animations with Manim & Remotion on GPU, merges everything with FFmpeg, runs QA, stores it on Cloudflare R2, and delivers it to students worldwide via CDN — **fully automated**.